

Fiserv 2026

A play-by-play of what we did to fix networking on the `Ubuntu` VM, the EdgeRouter, and the ESXi host's port-group layout. Includes commands, output highlights, and the reasoning at each decision point.

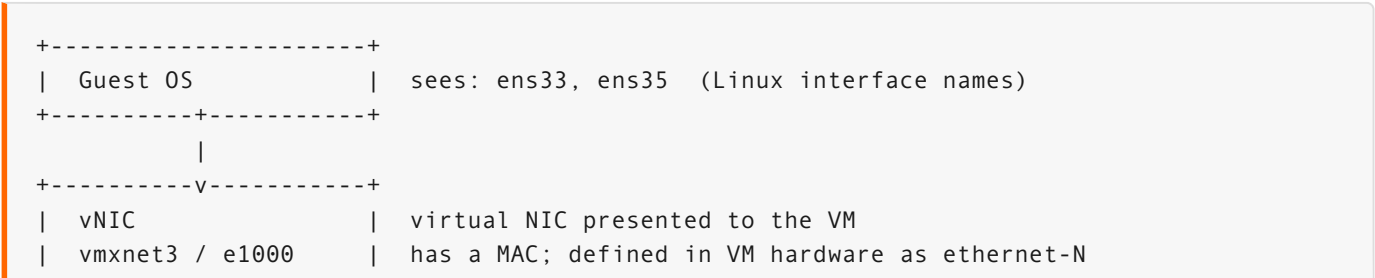
Environment

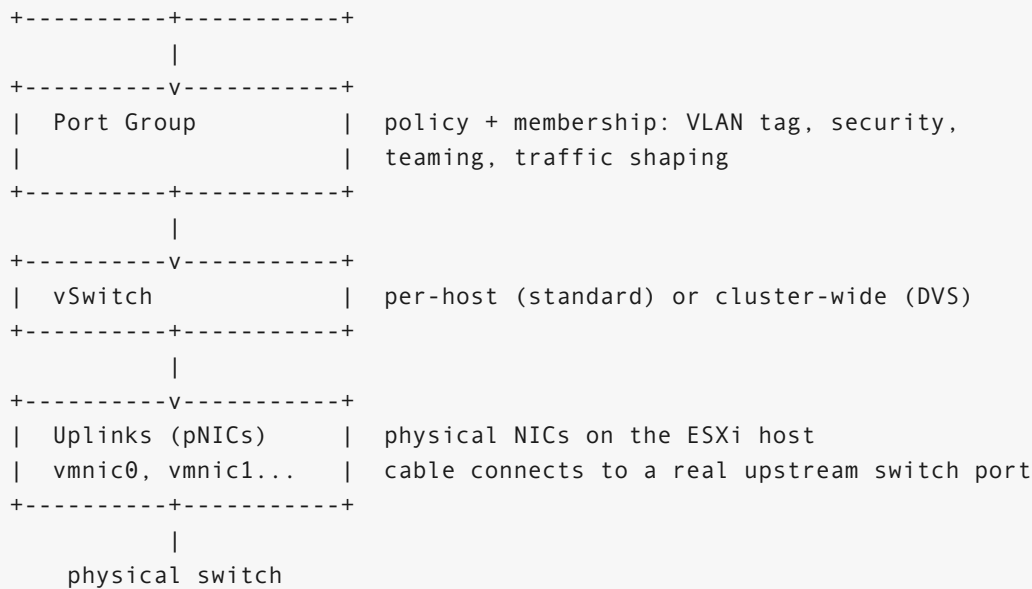
Item	Value
vCenter	<code>vcenter.skynetsystems.io</code> (vCenter Server 9.0.2 build 25148086)
ESXi host	<code>192.168.2.75</code> (Dell, Xeon Gold 6230, 256 GB RAM)
EdgeRouter	<code>192.168.1.1</code> (EdgeRouter 4, EdgeOS 2.0.9-hotfix.7)
Target VM	<code>Ubuntu</code> (Ubuntu 24.04.4 LTS, VMware Tools 12.5.0)
LAN1	192.168.1.0/24 — eth1 on EdgeRouter
LAN2	192.168.2.0/24 — eth2 on EdgeRouter, host's vmk0 lives here
LAN3	192.168.3.0/24 — eth3 on EdgeRouter
Tooling installed	<code>govc 0.54.0</code> , <code>expect</code> (already on macOS)

vSphere Networking — How the Pieces Fit

A short primer on the layers, because several issues today touched different levels of this stack.

The hierarchy





Layer by layer

Physical NICs (pNIC, `vmnicN`) Actual cards in the host, named `vmnic0`, `vmnic1`, etc. Each pNIC attaches to **exactly one** vSwitch as an uplink. Inspected with `esxcli network nic list`. The host in this lab has 8 of them.

Standard vSwitch A per-host virtual switch defined locally on each ESXi host. Simple, but config has to be replicated by hand on every host that needs the same port groups (or vMotion breaks). We created `vSwitch1` and `vSwitch2` today.

Distributed Virtual Switch (DVS / vDS) A cluster-wide virtual switch defined in vCenter, which pushes a consistent config to every member host. Required for features like network I/O control, cross-host LACP, and stable port-ID port binding. This lab uses `DSwitch` for management/vMotion/vSAN.

Port Group A named segment on a vSwitch. This is the unit of network policy: VLAN ID, security (promiscuous, MAC changes, forged transmits), teaming, shaping. Two flavors:

- **VM port group** — VMs attach their vNICs here. Example: `VM Network`, `VM Network 2`, `dswitch-vm`.
- **VMkernel port group** — the host's own services attach here. Each enabled service (management, vMotion, vSAN, iSCSI, NFS, replication) needs at least one vmk adapter on a port group with that role. Example: `dswitch-Management Network` (for `vmk0`), `dswitch-vmotion`.

DVS port groups carry an internal moref like `dvportgroup-25` (we saw that ID for `dswitch-Management Network` today).

VLAN at the port group level The port group sets the VLAN tag for traffic egressing its uplinks. VLAN ID 0 means untagged. The upstream switch port must agree (access port for one VLAN, or trunk passing the right tags). Every port group we touched today was VLAN 0.

VMkernel adapter (vmk) A host-level virtual NIC, separate from any VM. Each vmk has its own IP, attaches to one port group, and is bound to one or more services (management, vMotion, vSAN, NFS, etc.). vmk0 is conventional for management. Today's host has vmk0 = 192.168.2.75 on dswitch-Management Network.

Virtual NIC (vNIC) The NIC the guest OS sees. Comes in several driver flavors — VMXNET3 (paravirtual, recommended), E1000/E1000e (emulated), PVRDMA. Has a MAC assigned at creation. Lives in the VM as ethernet-N (the device key visible to govc device.ls).

How a packet flows

Outbound from a VM: 1. Guest OS writes the frame via its driver (vmxnet3 on Linux) 2. The vNIC delivers it into its port group 3. The vSwitch enforces port-group policy (security, VLAN tag) 4. The vSwitch picks an uplink per its teaming policy 5. The pNIC transmits onto the wire 6. The physical switch forwards based on VLAN/MAC

Inbound is the reverse, with the vSwitch using MAC learning to pick the right vNIC for delivery.

Common pitfalls (some of which we hit today)

- **Stealing a pNIC.** Adding a vmnic as an uplink on a new vSwitch removes it from whatever vSwitch it was on. This is how creating a new standard vSwitch with vmnic4 would have killed the DSwitch today.
- **Standard vSwitch and DVS coexist** on the same host, but each pNIC is owned by exactly one. We ended the day with DSwitch on vmnic4, vSwitch1 on vmnic5, vSwitch2 on vmnic6.
- **“No Network” in the UI** is an empty *Observed IP Ranges* hint, not an error. ESXi populates it from passive observation of broadcasts + CDP/LLDP; on a quiet uplink it stays blank.
- **VLAN 0 ≠ “no VLAN” upstream.** It means “egress untagged”. If the upstream switch port is configured as VLAN 99 access, traffic ends up on VLAN 99 anyway.
- **Predictable interface names depend on PCI slot.** Removing a vNIC and adding a new one can rename ens33 → ens35 and break a name-keyed netplan. Match by MAC instead.

Today's host networking at a glance

Layer	Object	What's on it
pNIC	vmnic4	

Layer	Object	What's on it
		DSwitch uplink (management, vMotion, vSAN, etc.)
pNIC	vmnic5	vSwitch1 uplink → EdgeRouter eth3 (192.168.3.0/24)
pNIC	vmnic6	vSwitch2 uplink
vSwitch	DSwitch (DVS)	hosts 5 DVS port groups
vSwitch	vSwitch1 (std)	hosts VM Network
vSwitch	vSwitch2 (std)	hosts VM Network 2
Port group	dswitch-Management Network	carries vmk0 (mgmt)
Port group	dswitch-vmotion	vMotion vmk
Port group	DSwitch-vSAN	vSAN vmk
Port group	VM Network	workload VMs, vmnic5 path
Port group	VM Network 2	workload VMs, vmnic6 path
vmk	vmk0	192.168.2.75 — host management
vNIC	Network adapter 1 (Ubuntu VM)	on VM Network, MAC 00:50:56:8e:b6:f3

1 — Ubuntu VM has no IP

Symptom

SSH to 192.168.1.125 (which had worked earlier in the day) was now timing out, and the user reported “my VM can’t get a network IP”.

Step 1.1 — Get into vCenter

govc wasn’t installed. Installed via Homebrew:

```
brew install govc          # 0.54.0
```

Connected with admin credentials. vCenter cert is self-signed, so `GOVC_INSECURE=true`:

```
export GOVC_URL='vcenter.skynetsystems.io' \  
GOVC_USERNAME='administrator@vsphere.local' \  
GOVC_PASSWORD='<vCenter-password>' \  
GOVC_INSECURE=true  
  
govc about  
# FullName: VMware vCenter Server 9.0.2 build-25148086
```

Step 1.2 — Locate the VM and inspect its NIC

```
govc find / -type m -name '*ubuntu*'\  
# /Datacenter/vm/Ubuntu  
  
govc vm.info -e -r /Datacenter/vm/Ubuntu | grep -iE 'ip|network|tools'\  
# IP address:      (empty)  
# Network:        dswitch-Management Network  
# tools:          toolsOk, guestToolsRunning
```

Tools was running but vCenter saw no IP. That's a strong signal the issue is inside the guest, not in the virtual networking layer. But to be sure, I dumped each NIC's full backing:

```
govc vm.info -json /Datacenter/vm/Ubuntu | python3 -c "  
import json, sys  
vm = json.load(sys.stdin)['virtualMachines'][0]  
for d in vm['config']['hardware']['device']:  
    if 'macAddress' in d:  
        b = d.get('backing', {})  
        net = b.get('deviceName') or b.get('port', {}).get('portgroupKey') or '?'  
        print(f"  label={d['deviceInfo']['label']} mac={d['macAddress']} backing={net}\n")  
"  
# label=Network adapter 2  mac=00:50:56:8e:55:60  backing=dvportgroup-25  
# net entries: 0
```

Two clues here: the label is “Network adapter 2” (no adapter 1 — it was removed) and Tools reports zero network entries. Together they pointed at a **NIC swap that broke the guest's interface naming**.

Step 1.3 — Confirm via guest ops

vCenter only saw the NIC topology; the guest OS was the source of truth for DHCP/netplan state.

VMware Tools' `guest.run` API lets us run commands inside the VM without needing IP-level access:

```

export GOVC_GUEST_LOGIN='ubuntu:<guest-password>'

govc guest.run -vm /Datacenter/vm/Ubuntu /usr/sbin/ip -br link
# lo      UNKNOWN    ...
# ens35   DOWN       00:50:56:8e:55:60

govc guest.run -vm /Datacenter/vm/Ubuntu /usr/bin/journalctl -u systemd-networkd --no-
pager -n 60
# May 06 04:35 ens33: DHCPv4 address 192.168.1.125/24 ... acquired from 192.168.1.1
# May 09 02:24 ens33: Lost carrier
# May 09 02:27 eth0 renamed to ens35

```

Diagnosis confirmed. The original NIC was named `ens33`, picked up 192.168.1.125 from DHCP. After the user removed adapter 1 and added a new one connected to the management network, the kernel named the new NIC `ens35` because predictable interface naming is derived from the PCI bus address. Cloud-init's netplan still configured `ens33` only, so `ens35` stayed DOWN with no DHCP attempt.

Step 1.4 — Side quest: get sudo working through guest.run

`/etc/netplan/50-cloud-init.yaml` is mode 600. To read it (and later write a new netplan file) we need sudo. The first attempt:

```

govc guest.run -vm $VM /bin/sh -c 'echo <guest-password> | sudo -S id'
# sudo: no password was provided

```

Then with a password file:

```

govc guest.run -vm $VM /bin/sh -c 'umask 077; printf <guest-password>\n > /tmp/.pw'
govc guest.run -vm $VM /bin/sh -c 'sudo -S id < /tmp/.pw'
# usage: sudo -h | -K | -k | -V      (sudo got bad args)

```

And a probe of pipe behavior:

```

govc guest.run -vm $VM /bin/sh -c 'printf <guest-password> | xxd | head -2'
# (empty output)

```

So `vmtoolsd`'s `RunProgramInGuest` **doesn't reliably propagate stdout between processes in a pipe**, and `< redirect` on a single sh `-c` line got mangled. The workaround that finally worked was a **here doc** — here doc input is part of the script body, so it doesn't depend on inter-process plumbing:

```

cat > /tmp/setup2.sh <<'OUTER'
#!/bin/bash
sudo -S sh -c '

```

```

    echo "ubuntu ALL=(ALL) NOPASSWD: ALL" > /etc/sudoers.d/90-ubuntu-nopasswd
    chmod 440 /etc/sudoers.d/90-ubuntu-nopasswd
' <<EOF
<guest-password>
EOF
OUTER

govc guest.upload -f -vm $VM /tmp/setup2.sh /tmp/setup2.sh
govc guest.run -vm $VM /bin/chmod +x /tmp/setup2.sh
govc guest.run -vm $VM /tmp/setup2.sh
# OK_NOPASSWD_INSTALLED

```

After this, `sudo -n` works for the `ubuntu` user. **Lesson:** when running non-trivial shell through `guest.run`, prefer scripts uploaded via `guest.upload` over inline `sh -c` chains, and use heredocs to feed stdin rather than pipes.

Step 1.5 — Fix netplan

Rather than patch `50-cloud-init.yaml` (cloud-init regenerates it), I added a higher-numbered file that **matches by MAC address**. Matching by MAC survives interface renames; matching by name is what just broke.

```

cat > /tmp/netplan-fix.sh <<'EOF'
#!/bin/bash
sudo tee /etc/netplan/60-ens35.yaml >/dev/null <<'YAML'
network:
  version: 2
  ethernets:
    primary:
      match:
        macaddress: 00:50:56:8e:55:60
      dhcp4: true
      dhcp6: false
YAML
sudo chmod 600 /etc/netplan/60-ens35.yaml
sudo netplan apply
ip -br addr
EOF

govc guest.upload -f -vm $VM /tmp/netplan-fix.sh /tmp/netplan-fix.sh
govc guest.run -vm $VM /bin/chmod +x /tmp/netplan-fix.sh
govc guest.run -vm $VM /tmp/netplan-fix.sh
# ens35 UP 192.168.2.220/24 metric 100

```

Verified L3 reachability:

```
govc guest.run -vm $VM /usr/bin/ping -c 3 192.168.2.1      # gateway: ok
govc guest.run -vm $VM /usr/bin/getent hosts google.com    # DNS: ok
ssh ubuntu@192.168.2.220 'hostname'                       # workstation reach: ok
```

Outcome

VM now has a stable IPv4 lease (`192.168.2.220`) on the dswitch-Management Network port group, and the netplan config is robust against future NIC renames.

2 — Build a separate “VM Network” port group

Goal

Create a workload-only port group on the host, isolated from the management DVS, with its own physical uplink. This gives a place to attach experimental VMs that won't share L2 with management/vMotion/vSAN.

Step 2.1 — Find a free pNIC

```
govc host.esxcli -host /Datacenter/host/Cluster/192.168.2.75 -- network nic list
# vmnic0-3    Up admin, Down link (unconnected)
# vmnic4      Up admin, Up    1 Gbps ← DSwitch uplink (do not touch)
# vmnic5      Up admin, Up    1 Gbps ← UNCLAIMED
# vmnic6      Up admin, Up    1 Gbps ← UNCLAIMED
# vmnic7      Up admin, Down link

govc host.esxcli -host $HOST -- network vswitch dvs vmware list | grep Uplinks
# Uplinks: vmnic4
```

Step 2.2 — Create the vSwitch and port group

`vmnic5` was the natural choice (link-up, unclaimed, lowest free).

```
govc host.vswitch.add -host $HOST -nic vmnic5 vSwitch1
govc host.portgroup.add -host $HOST -vswitch vSwitch1 -vlan 0 'VM Network'

govc host.vswitch.info -host $HOST
# Name: vSwitch1    Portgroup: VM Network    Pnic: vmnic5
```


Step 2.3 — Diagnose the “No Network” UI label

The vSphere Client showed `vmnic5` as `No Network`. That field is **Observed IP Ranges**, populated passively from L2 traffic. Confirmed the link is healthy — receive counters show frames are arriving:

```
govc host.esxcli -host $HOST -- network nic stats get -n vmnic5
# Broadcastpacketsreceived: 556
# Bytesreceived:          76188
# Receive*errors:         0    (across CRC, FIFO, frame, length, missed, over)
```

So the cable + upstream switch port + driver are all fine. “No Network” was purely cosmetic — ESXi simply hadn’t built up an observation window yet. CDP was in `listen` mode by default so no neighbor info was advertised either.

3 — Hot-add a second NIC on “VM Network”, debug DHCP

Step 3.1 — Add the NIC

```
govc vm.network.add -vm /Datacenter/vm/Ubuntu -net 'VM Network' -net.adapter vmxnet3
```

In-guest the new NIC came up named `ens33` — the same name the original NIC had used, because the new VMXNET3 landed at the old PCI bus position. Fortunate, because `50-cloud-init.yaml` still had `ens33: dhcp4: true`. But after `netplan apply`:

```
ens33    UP    fe80::250:56ff:fe8e:b6f3/64    ← link-local IPv6 only, no IPv4
```

L2 was up, but DHCP wasn’t completing. The system journal showed the DHCPv4 request was being made, but no offers were arriving.

Step 3.2 — Decide where to look next

Two halves to a DHCP exchange. Ours was sending DISCOVER but receiving no OFFER. Two diagnostic angles:

1. **Are the discovers reaching anyone?** Check pNIC TX/RX deltas.
2. **Who would have answered?** Investigate the EdgeRouter, since it owns all the DHCP scopes for these subnets.

I started with (1) because it’s quick and proves whether the L2 path is even intact:

```
# vmnic5 stats during a DHCP attempt
govc host.esxcli -host $HOST -- network nic stats get -n vmnic5
# Bytesent went from 13004 → 18972 (+5968 bytes, ~6 KB of DHCP discovers)
# Bytesreceived stayed at 76188 (zero new packets back)
```

ESXi was sending discovers; nothing was responding. Next step: investigate the router.

4 — EdgeRouter: why isn't LAN3 serving DHCP

Step 4.1 — Connect

EdgeOS uses Vyatta-style CLI with mandatory password auth. Drove it via `expect`:

```
spawn ssh admin@192.168.1.1
expect "assword:"
send "<vCenter-password>\r"
expect "$ "
```

Step 4.2 — Map interfaces and DHCP scopes

```
show interfaces
# eth0 136.47.236.42/20 Internet (WAN)
# eth1 192.168.1.1/24 Local 1
# eth2 192.168.2.1/24 Local 2
# eth3 192.168.3.1/24 Local 3

configure
show service dhcp-server | no-more
# shared-network-name LAN1 { authoritative enable; subnet 192.168.1.0/24 ... }
# shared-network-name LAN2 { authoritative enable; subnet 192.168.2.0/24 ... }
# shared-network-name LAN3 {                               subnet 192.168.3.0/24 ... }
```

Two visible deltas between LAN3 and the working scopes:

1. **LAN3 is missing** `authoritative enable`
2. LAN3 is missing `lease 86400` (other LANs set 86400s)

I initially suspected `authoritative` was the issue, but ISC dhcpd's `authoritative` only controls whether dhcpd issues DHCPNAK to misconfigured clients — it shouldn't prevent DHCPDISCOVER → DHCPOFFER. So I held off on that hypothesis and looked further.

Step 4.3 — Verify DHCP traffic actually reaches eth3

I needed to prove that `vmnic5 → ?` actually terminates at `eth3` and not somewhere else. Two snapshots of the eth3 packet counters bracketing a fresh DHCPDISCOVER from the VM:

```
# On the router, before:
show interfaces ethernet eth3 | no-more
# RX: 25089 bytes / 209 packets / 102 dropped / 61 mcast
# TX: 1541106 bytes / 32407 packets

# Then trigger DHCP on the VM:
govc guest.run -l 'ubuntu:<guest-password>' -vm $VM \
  /usr/bin/sudo /usr/bin/networkctl reconfigure ens33

# On the router, after:
show interfaces ethernet eth3 | no-more
# RX: 26667 bytes / 216 packets / 102 dropped / 64 mcast    ← +7 packets, +1578 bytes
# TX: 1541214 bytes / 32409 packets                        ← +2 packets, +108 bytes
```

RX grew by 7 packets — DHCPDISCOVER definitely reached the router on eth3. **TX grew by only 2 small packets** (~54 bytes each — ARP-sized, not DHCP-sized: a real DHCPOFFER is 342+ bytes). So dhcpd received the discover and chose not to respond.

Step 4.4 — Check whether dhcpd is actually bound to eth3

```
ps -ef | grep dhcpd3 | grep -v grep
# root  2065  /usr/sbin/dhcpd3 -pf /var/run/dhcpd.pid -cf /opt/vyatta/etc/dhcpd.conf -lf /
      var/run/dhcpd.leases

sudo netstat -nlup | grep ':67'
# udp   0    0  0.0.0.0:67  0.0.0.0:*  2065/dhcpd3    ← listening on all
```

dhcpd was listening on `0.0.0.0:67`, so the kernel was delivering packets to it. The filtering had to be at the application layer. Searched the syslog for dhcpd messages:

```
sudo cat /var/log/messages | grep dhcpd3 | tail -20
# May  1 13:34:42 dhcpd3: No subnet declaration for eth3 (no IPv4 addresses).
# May  1 13:34:42 dhcpd3: ** Ignoring requests on eth3.  If this is not what
# May  1 13:34:42 dhcpd3:    you want, please write a subnet declaration
# May  1 13:34:42 dhcpd3:    in your dhcpd.conf file for the network segment
# May  1 13:34:42 dhcpd3:    to which interface eth3 is attached. **
```

Smoking gun. When dhcpd3 started on May 1 at 13:34, **eth3 had no IPv4 address yet**, so dhcpd marked eth3 as “ignore” for the lifetime of that process. The IP came later, the LAN3 scope is now correct, but dhcpd never re-evaluated.

This is a classic ISC-dhcpd race — the daemon caches the interface→subnet mapping at startup. The symptom is binary (works or doesn't) and survives config changes. The `authoritative` red herring earlier was real config hygiene drift but was not what was breaking DHCP.

Step 4.5 — Restart dhcpd

EdgeOS's expected operational command (`restart dhcp server`) didn't exist on this firmware version:

```
restart dhcp          # Invalid command
restart dhcp-server   # Invalid command
restart service dhcp-server # Invalid command
```

The init script worked:

```
ls /etc/init.d/ | grep -i dhcp
# dhcpd
# dhcpdv6
# vyatta-dhcp3-relay
# vyatta-dhcp3-server

sudo /etc/init.d/dhcpd restart

ps -ef | grep dhcpd3
# root  29710  /usr/sbin/dhcpd3 ...    ← new PID

sudo tail /var/log/messages | grep dhcpd3
# May  9 04:11:26 dhcpd3: No subnet declaration for eth0 (136.47.236.42).
# (no "Ignoring requests on eth3" message anymore)
```

Restart succeeded. The new dhcpd3 startup log only complains about eth0 (the WAN port — correct, no DHCP scope there). eth3 was bound this time.

Step 4.6 — Verify the lease

```
govc guest.run -l 'ubuntu:<guest-password>' -vm $VM \
  /usr/bin/sudo /usr/bin/networkctl reconfigure ens33

govc guest.run -l 'ubuntu:<guest-password>' -vm $VM /usr/sbin/ip -br addr
# ens33  UP  192.168.3.4/24 metric 100

# On the router:
show dhcp leases
# 192.168.3.4  00:50:56:8e:b6:f3  2026/05/10 04:11:47  LAN3  ubuntu
```

Outcome

LAN3 DHCP works. The VM has a stable lease on 192.168.3.0/24 via eth3.

Worth noting for the future

The dhcpd-vs-eth3 race could happen again if the router reboots and eth3's IP comes up after dhcpd. Two robust mitigations (didn't apply yet, just noted):

1. Add `post-up sleep N && service dhcpd restart` to the eth3 ifupdown sequence
2. Cron `@reboot sleep 60 && /etc/init.d/dhcpd restart`

Also fix the config drift: add `authoritative enable` to LAN3 to match the other shared-networks. Strictly cosmetic right now.

5 — Drop the redundant NIC

Goal

With LAN3 working, the dual-homed setup wasn't needed. The user asked for the management-network NIC to be removed so the VM is single-homed on `VM Network`.

Risk assessment before removing

- We're managing the VM via VMware Tools (govc guest.run). That uses the VMCI device, **not** any network adapter. Removing the NIC won't break Tools-based access.
- The Graylog UI/API and syslog input were bound to `0.0.0.0`, so they follow whichever IP the VM has. After this change, they live at `192.168.3.4` instead of `192.168.2.220`.

Step 5.1 — Identify the right device key

```
govc vm.info -json /Datacenter/vm/Ubuntu | python3 -c "
import json, sys
vm = json.load(sys.stdin)['virtualMachines'][0]
for d in vm['config']['hardware']['device']:
    if 'macAddress' in d:
        print(d['key'], d['deviceInfo']['label'], d['macAddress'],
              d.get('backing', {}).get('deviceName') or
              d.get('backing', {}).get('port', {}).get('portgroupKey'))
"
# 4001 Network adapter 2 00:50:56:8e:55:60 dvportgroup-25 ← REMOVE
# 4000 Network adapter 1 00:50:56:8e:b6:f3 VM Network ← KEEP
```

```
govc device.ls -vm /Datacenter/vm/Ubuntu | grep ethernet
# ethernet-1 VirtualVmxnet3 DVSwitch:... ← maps to key 4001
# ethernet-0 VirtualVmxnet3 VM Network ← maps to key 4000
```

Step 5.2 — Remove

```
govc device.remove -vm /Datacenter/vm/Ubuntu ethernet-1
```

Step 5.3 — Clean up the orphan netplan file

`60-ens35.yaml` matched the now-removed MAC. It would just sit idle, but clutter is clutter:

```
govc guest.run -l 'ubuntu:<guest-password>' -vm $VM \
  /usr/bin/sudo /bin/rm -f /etc/netplan/60-ens35.yaml
govc guest.run -l 'ubuntu:<guest-password>' -vm $VM \
  /usr/bin/sudo /usr/sbin/netplan apply
```

Step 5.4 — Verify

```
govc guest.run -l 'ubuntu:<guest-password>' -vm $VM /usr/sbin/ip -br addr
# lo UNKNOWN 127.0.0.1/8 ::1/128
# ens33 UP 192.168.3.4/24 metric 100

govc guest.run -l 'ubuntu:<guest-password>' -vm $VM /usr/sbin/ip route
# default via 192.168.3.1 dev ens33 ... ← single default, no more conflict
```

Outcome

VM is single-homed on `VM Network` (192.168.3.4). The earlier two-default- route ambiguity is gone.

6 — Add a second isolated port group “VM Network 2”

The dangerous request

Ask was: “create another network named ‘VM Network 2’ that’s connected to `vmnic4`”. This would have been catastrophic.

Why pausing mattered

`vmnic4` is the **only uplink for the DSwitch**. The DSwitch carries:

- `dswitch-Management Network` — the host's vmk0 (192.168.2.75) — the path vCenter uses to reach the host
- `dswitch-vmotion`
- `dswitch-VM Network-ephemeral`
- `dswitch-vm`
- `DSwitch-vSAN`

Creating a standard vSwitch with `vmnic4` as its uplink reassigns the physical NIC. The DSwitch loses its only uplink in the same instant. That would have:

1. Disconnected the host from vCenter (vmk0 has no path)
2. Stopped vMotion mid-migration (if any was happening)
3. Killed vSAN traffic
4. Black-holed every VM currently on a DVS port group

Recovery would require **physical/iLO console access to the ESXi host**.

So I stopped, listed the available pNICs, and offered alternatives: - `vmnic6` — link-up at 1 Gbps, completely unclaimed → natural fit - Add the port group on the existing DSwitch → no new vSwitch, shares the vmnic4 uplink with management - Use `vmnic4` anyway, accept the blast radius

The user picked `vmnic6`.

Step 6.1 — Create

```
govc host.vswitch.add -host $HOST -nic vmnic6 vSwitch2
govc host.portgroup.add -host $HOST -vswitch vSwitch2 -vlan 0 'VM Network 2'

govc find / -type n
# /Datacenter/network/VM Network 2      ← new
# /Datacenter/network/VM Network
# /Datacenter/network/dswitch-Management Network
# /Datacenter/network/dswitch-vmotion
# /Datacenter/network/dswitch-vm
# /Datacenter/network/dswitch-VM Network-ephemeral
# /Datacenter/network/DSwitch-vSAN
# /Datacenter/network/DSwitch-DVUplinks-21
# /Datacenter/network/test
```

Outcome

Three VM-facing port groups now exist:

Port group	vSwitch	pNIC	Notes
(DVS port groups)	DSwitch	vmnic4	management / vMotion / vSAN
VM Network	vSwitch1	vmnic5	EdgeRouter eth3 → 192.168.3.0/24
VM Network 2	vSwitch2	vmnic6	wherever vmnic6 is cabled

Final state of the VM

```
Name:          Ubuntu
Host:          /Datacenter/host/Cluster/192.168.2.75
NICs:          1
  Network adapter 1 mac=00:50:56:8e:b6:f3 on 'VM Network' (vmnic5 → eth3)
Guest IP:       192.168.3.4
Routes:        default via 192.168.3.1 dev ens33
Sudo:          passwordless for ubuntu user (sudoers.d/90-ubuntu-nopasswd)
Netplan:
  /etc/netplan/50-cloud-init.yaml (cloud-init default — references
                                   ens33 by name, harmless on this VM
                                   because the new NIC came up as ens33)
```

Reusable patterns that came out of this session

Pattern: prefer MAC-match over name-match in netplan

When a NIC is replaced, predictable interface names change with the PCI slot. A MAC-match in netplan survives renames. Cloud-init's `50-cloud-init.yaml` uses name match, which is the wrong default for any machine whose NIC topology might change.

Pattern: getting around `vmtoolsd`'s `RunProgramInGuest` quirks

Pipes and stdin redirects between processes don't reliably propagate. Instead:

1. Write the full script locally
2. `govc guest.upload` it to the VM

3. `govc guest.run /bin/chmod +x` then run it

Use heredocs inside the script to feed stdin (e.g. for `sudo -S`); avoid relying on multi-process pipes.

Pattern: prove the L2 path before debugging L3

Before assuming a service is broken, prove the packet is reaching it. Cheap to do, decisively answers the binary question of “is this an L2 problem or an application problem”. For DHCP that means snapshot pNIC counters or interface counters before/after a fresh request.

Pattern: confirm scope before destructive vSwitch changes

Standard vSwitch creation re-assigns its specified uplink, regardless of prior owner. Always cross-check `Uplinks:` on every existing standard switch *and* on every DVS before adding a new one. The blast radius of hijacking the wrong pNIC is total host disconnect.

Reference: handy commands from the session

govc

```
# auth
export GOVC_URL='vcenter.skynetsystems.io'
export GOVC_USERNAME='administrator@vsphere.local'
export GOVC_PASSWORD='<vCenter-password>'
export GOVC_INSECURE=true
export GOVC_GUEST_LOGIN='ubuntu:<guest-password>'

# discover
govc about
govc datacenter.info
govc find / -type m           # all VMs
govc find / -type n           # all networks/port groups
govc find / -type h           # all hosts

# VM detail
govc vm.info -e -r /path/to/VM
govc vm.info -json /path/to/VM # full structured dump
govc device.ls -vm /path/to/VM # devices with internal names

# host networking
govc host.esxcli -host $HOST -- network nic list
govc host.esxcli -host $HOST -- network nic stats get -n vmnicN
govc host.esxcli -host $HOST -- network vswitch standard list
govc host.esxcli -host $HOST -- network vswitch dvs vmware list
govc host.vswitch.info -host $HOST
```

```
govc host.portgroup.info -host $HOST
govc host.vnic.info -host $HOST      # vmkernel adapters

# create / modify / remove
govc host.vswitch.add -host $HOST -nic vmnicN vSwitchName
govc host.portgroup.add -host $HOST -vswitch vSwitchName -vlan 0 'PG name'
govc vm.network.add -vm $VM -net 'PG name' -net.adapter vmxnet3
govc device.remove -vm $VM ethernet-N

# guest ops
govc guest.run -vm $VM /path/to/program arg1 arg2
govc guest.upload -f -vm $VM ./local /tmp/remote
```

EdgeOS via expect

```
#!/usr/bin/expect -f
set timeout 20
spawn ssh admin@192.168.1.1
expect "assword:"
send "<vCenter-password>\r"
expect "$ "
send "show service dhcp-server | no-more\r" ; # bypass pager in configure mode
expect "# "
send "exit\r"
```

Useful EdgeOS commands:

```
show interfaces
show interfaces ethernet ethN | no-more
show vlan
show dhcp leases
configure
show service dhcp-server | no-more
show firewall | no-more
exit
```

For service restart on this firmware (the operational `restart` verb doesn't include dhcp):

```
sudo /etc/init.d/dhcpd restart
```

Quick Reference — Critical Commands

Run-on-the-device commands that mattered most during this session. None of these go through vCenter / govc — they're what you'd type on a shell in the guest, or on the EdgeRouter via SSH.

Inside the Ubuntu VM

```
# Interface state
ip -br link
ip -br addr
ip route

# Force a fresh DHCPDISCOVER on a specific interface
sudo networkctl reconfigure ens33

# Apply netplan changes
sudo netplan apply

# What netplan generated for systemd-networkd
sudo ls /run/systemd/network/

# networkd's recent activity (look for DHCP success/fail, carrier events)
sudo journalctl -u systemd-networkd --no-pager -n 60
sudo networkctl status ens33

# Add a MAC-matched DHCP netplan stanza (robust against NIC rename)
sudo tee /etc/netplan/60-mynic.yaml >/dev/null <<'YAML'
network:
  version: 2
  ethernets:
    primary:
      match:
        macaddress: 00:50:56:8e:b6:f3
      dhcp4: true
      dhcp6: false
YAML
sudo chmod 600 /etc/netplan/60-mynic.yaml
sudo netplan apply

# Make the ubuntu user passwordless for sudo (one-time)
echo 'ubuntu ALL=(ALL) NOPASSWD: ALL' | sudo tee /etc/sudoers.d/90-ubuntu-nopasswd
sudo chmod 440 /etc/sudoers.d/90-ubuntu-nopasswd
```

On the EdgeRouter (`ssh admin@192.168.1.1`)

```
# Interface IPs, link states
show interfaces
show interfaces ethernet eth3 | no-more

# Current DHCP leases (operational mode)
show dhcp leases

# DHCP server config (must be in configure mode; pager bypassed)
configure
```

```

show service dhcp-server | no-more
exit

# Inspect the dhcpd3 process and listening sockets
ps -ef | grep dhcpd3 | grep -v grep
sudo netstat -nlup | grep ':67'

# Look for "Ignoring requests on ethN" — startup race indicator
sudo cat /var/log/messages | grep dhcpd3 | tail -30

# Restart dhcpd (the EdgeOS `restart dhcp ...` op-mode verb does NOT exist
# on EdgeOS 2.0.x — use the init script directly)
sudo /etc/init.d/dhcpd restart

# Capture DHCP packets on an interface to a pcap file, then read it
sudo tcpdump -ni eth3 -c 6 'udp port 67 or udp port 68' -w /tmp/dhcp.pcap
sudo tcpdump -nr /tmp/dhcp.pcap

# Per-interface RX/TX counter snapshot (use before/after to prove
# whether traffic actually traversed an interface)
show interfaces ethernet eth3 | no-more

```

Playbook: “DHCP isn’t working”

Run these in order; first one that fails tells you where the break is.

1. **Client interface up?** — `ip -br link` on the VM → state `UP`
2. **Client is sending DISCOVER?** — `sudo journalctl -u systemd-networkd -n 30` should show DHCPv4 attempt lines after a `networkctl reconfigure`
3. **DISCOVER reaches the router?** — snapshot `show interfaces ethernet ethN` RX packets, run a `networkctl reconfigure`, snapshot again. RX should grow by a few packets.
4. **Router sent OFFER back?** — TX counter delta on same interface should grow by similar magnitude (~342 bytes per OFFER). If RX grew but TX didn’t, dhcpd ignored the request.
5. **Why is dhcpd ignoring it?** — `sudo cat /var/log/messages | grep dhcpd3` for `No subnet declaration for ethN (no IPv4 addresses)` or `Ignoring requests on ethN` — that’s the race-on-startup case. Fix: `sudo /etc/init.d/dhcpd restart`.
6. **dhcpd answered but client didn’t take it?** — `sudo tcpdump -ni ethN -c 6 'udp port 67 or 68'` shows the full conversation; check the client log for `DHCPv4 lease lost` or NAK.